

基于多目标的异形圆管切割与余料优化研究

摘 要

飞机巡航过程受到飞行高度、空速、大气密度、温度、风场、飞机质量及发动机推力效率等多种因素共同影响。因此，仅凭经验策略难以获得全局最优巡航方案，需要建立高度与速度联合优化的数学模型，对巡航过程进行定量分析。题目要求围绕某型民用客机的巡航性能优化，完成建模、推导、数值计算与模型扩展等任务。

针对问题一，分析商品买卖机理，**建立基于双十一规则商家利益最大化判别模型**。本文以经济学价格理论为基础，分析得商家参与活动通过降低单次商品利润提高总销量而获得更高利润。**选取利润为指标**，基于商家参与活动规则，综合活动前后利润判别式，以营业额最大化为目标建立判别模型。通过具体数据如 WIS 水润面膜相对非双十一销售额翻了近 110 倍说明商家是理性的。

针对问题三，分析选取消费者与商家群体双赢指标，**建立消费者是否购买与商家是否提供高质量服务的演化博弈模型**。本文考虑商家与消费者对“赢”的定义不同，查阅文献以**商家能吸引消费者购买和消费者能得到高质量服务为双赢指标**。基于不同行为下商家与消费者收益变化，建立演化博弈模型。**模型求解结果为消费者趋向于购买和商家趋向于提供高质量服务的双赢局面**，并通过 2009-2019 年成交总额的指数型上涨说明结果准确性。

针对问题三，分析选取消费者与商家群体双赢指标，**建立消费者是否购买与商家是否提供高质量服务的演化博弈模型**。本文考虑商家与消费者对“赢”的定义不同，查阅文献以**商家能吸引消费者购买和消费者能得到高质量服务为双赢指标**。基于不同行为下商家与消费者收益变化，建立演化博弈模型。**模型求解结果为消费者趋向于购买和商家趋向于提供高质量服务的双赢局面**，并通过 2009-2019 年成交总额的指数型上涨说明结果准确性。

针对问题四，基于商家竞争规律，**建立商家是否提供高质量商品的演化博弈模型**。考虑到电子商务普遍有商品质量不高现象，本文以**双十一作用下商家间竞争能否提高商品质量为指标**。基于不同行为下同类商家收益变化建立模型。**模型求解结果为同类商家之间趋向用高质量商品进行竞争**，通过 2013-2015 年双十一过后的投诉率 (6.9 、 6.1 、 4.6×10^{-7}) 不断下降说明结果准确性。

针对问题五：本文从商家和买家的角度考虑，在前面四问的基础上，以国家质检局抽检网络产品不合格率以及双十一个人网购数据为例对双十一购物期间某些不理性的行为进行说明，并提出“**理性消费**”的合理建议，即：**商家销售保持诚信、买家购物理性消费、买家积极反馈维权**。

关键词： 双十一购物节，基于目标规划的判别模型，演化博弈模型

1 问题重述

1.1 问题背景

在全球航空燃油价格波动、碳排放约束以及航空公司运营成本压力不断增大的背景下，如何通过科学的巡航策略降低燃油消耗，已成为飞行运行管理和飞机性能优化中的重要问题。

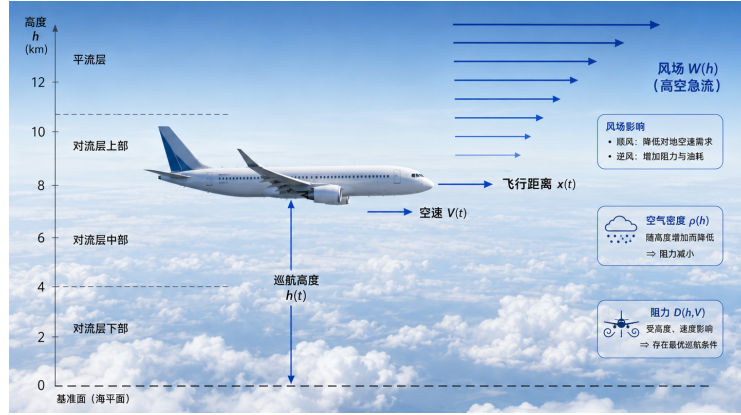


图 1: 飞机飞行状态示意图

1.2 问题提出

基于题目给定的飞机参数、初始状态和终止质量条件，本文需要依次解决以下问题。

问题一：根据气动力平衡和能量守恒原理，建立飞机巡航过程中水平位移、飞机质量、高度和空速之间的耦合动力学模型。并进一步推导等速巡航爬升策略和等马赫数巡航爬升策略下的高度演化规律，比较两种策略的高度剖面、质量剖面、爬升率、总飞行时间和燃油消耗量。

问题二：建立沿飞行路径积分的总燃油消耗模型，明确标准大气中温度、密度和声速随高度连续变化对油耗模型的影响。进一步考虑实际大气温度偏差，选择合理的修正方法，建立修正后的油耗率模型，并分析温度偏差对总油耗计算结果和系统非线性结构的影响。

问题三：在给定初始飞行状态下，以总燃油消耗最小为目标，建立高度-速度联合最优巡航控制模型。并通过合适的最优控制方法推导最优轨迹满足的必要条件，比较无风与有垂直风切变条件下最优解结构的变化。

问题四：在前三问基础上，建立完整的巡航策略综合数值计算模型。定量比较等速巡航爬升策略和考虑风场条件下的高度-速度联合轨迹两者燃油消耗、航程变化和节油比例；同时分析速度偏离惩罚系数变化对最优轨迹的影响，并从温度偏差修正、小角度非水平爬升、发动机安装损失、多机协同与空域约束等方向中选择至少两个进行模型扩展。

2 问题分析

2.1 问题一的分析

问题一要求建立飞机巡航阶段的动力学模型，并分析等速巡航爬升和等马赫数巡航爬升两种典型策略。巡航过程中，飞机质量因燃油消耗不断减小，而高度、空速、大气密度及气动力之间又存在较强耦合关系，因此首先需要依据飞行动力学原理建立巡航过程的数学模型。随后，在不同控制策略下对模型进行适当简化，分别推导两种巡航方式的高度变化规律，并结合题目给定参数进行数值计算，对比两种策略在爬升率、飞行时间及燃油消耗等方面的差异，为后续优化提供基础模型。

2.2 问题二的分析

问题二在问题一动力学模型的基础上，进一步关注巡航全过程的燃油消耗计算。由于飞机飞行过程中所处的大气环境随高度连续变化，大气密度、温度和声速均会影响气动力及发动机效率，因此需要建立能够反映大气参数变化影响的燃油消耗模型。同时，针对实际飞行中可能出现的温度偏差，需要建立修正模型，并通过数值计算分析温度修正对燃油消耗结果的影响程度，从而判断不同大气条件下模型的适用范围及修正的必要性。

2.3 问题三的分析

问题三要求在满足飞行动力学约束、升力平衡及飞行包线限制的条件下，以总燃油消耗最小为目标，建立高度与速度联合优化模型。由于控制变量与状态变量之间存在明显的非线性耦合关系，该问题本质上属于连续最优控制问题。建模过程中需要综合考虑高度变化、速度调整及风场影响对燃油消耗的共同作用，并选择合适的优化方法进行求解。同时，通过比较无风和有风条件下最优轨迹的变化规律，分析风场对最优巡航策略的影响，并评价不同求解方法在复杂约束条件下的适用性。

2.4 问题四的分析

问题四是在前三问基础上的综合应用与模型扩展。首先，需要建立完整的数值计算流程，对等速巡航策略和最优巡航策略进行统一计算与比较，分析最优策略在燃油节省、飞行时间及航程等方面的优势。随后，通过改变模型参数分析最优轨迹对参数扰动的敏感性，识别影响巡航性能的关键因素。最后，结合实际飞行环境，从温度偏差、小角度非水平飞行、发动机安装损失、多机协同等方面对模型进行扩展，使模型更加符合实际工程应用，提高模型的通用性和实际参考价值。

3 模型假设与符号说明

3.1 模型假设

1. 飞机巡航阶段可视为小角度爬升过程，航迹角较小，故有 $\cos \gamma \approx 1$ ；
2. 飞机在巡航过程中保持准定常飞行状态，升力近似平衡飞机重量；
3. 除燃油消耗外，飞机其他载荷质量保持不变，因此飞机质量变化仅由燃油消耗引起；
4. 风场仅影响飞机相对于地面的水平速度，不改变飞机相对于空气的空速；
5. 在基准模型中，采用指数大气模型描述大气密度随高度的变化。

3.2 符号说明

表 1: 主要符号说明

符号	说明	单位
$x(t)$	飞机相对于地面的累计飞行距离	
$m(t)$	飞机实时总质量（含剩余燃油	
$h(t)$	飞机巡航高度	
$V(t)$	飞机空速	
$M(t)$	飞机马赫数	—
$F(t)$	发动机推力	N
$D(t)$	飞机气动阻力	N
$\dot{m}_f(t)$	燃油消耗率	kg/s
$C_L(t)$	升力系数	—
$\rho(h(t))$	高度 $h(t)$ 处的大气密度	kg/m ³
$a(h)$	高度 h 处的声速	m/s
$W(h)$	高度 h 处的水场风速	m/s
$q(x)$	单位航程燃油消耗率	kg/m
J	总燃油消耗量	kg

4 模型准备

4.1 燃油消耗率

根据推力燃油消耗率 (TSFC) 的定义, 单位时间内的燃油消耗量与发动机推力需求近似成正比, 由此可建立基础燃油消耗率模型:

$$\dot{m}_f(t) = c_f F(t), \quad (1)$$

其中, c_f 表示推力燃油消耗系数, $F(t)$ 表示发动机在 t 时刻的推力需求。

在实际巡航过程中, 发动机燃油效率除受推力影响外, 还与飞行速度密切相关。研究表明, 巡航速度的选择会显著改变燃油消耗, 且存在使油耗最低的经济巡航速度, 该最优速度通常取决于飞行高度、飞机质量、航程及运行成本等因素 [?]. 为刻画这一特性, 本文在基础模型上引入速度偏离惩罚项, 用以描述空速 $V(t)$ 偏离经济速度 V_{opt} 时所产生的额外油耗:

$$\dot{m}_f(t) = c_f F(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right], \quad (2)$$

其中, V_{opt} 表示参考最佳经济巡航速度, β 表示速度偏离惩罚系数。

4.2 大气密度

巡航高度变化时, 大气密度随之改变, 进而影响飞机的升力、阻力及推力需求。根据标准大气特性, 本文采用指数衰减模型 [?] 描述密度随高度的变化:

$$\rho(h) = \rho_0 e^{-h/H_\rho}, \quad (3)$$

其中, ρ_0 表示海平面大气密度, H_ρ 表示大气标高。

4.3 马赫数与声速

声速依赖于大气温度, 采用国际标准大气的线性温度模型:

$$T(h) = T_0 - L_T h, \quad (4)$$

其中, T_0 为海平面标准温度, L_T 为温度递减率。由此声速可表示为:

$$a(h) = \sqrt{\gamma R_{\text{gas}} T(h)}. \quad (5)$$

其中 γ 表示空气的比热比, R_{gas} 表示气体常数, 则马赫数可表示为:

$$M(t) = \frac{V(t)}{a(h(t))}. \quad (6)$$

4.4 风场

题目给定的风场仅依赖于飞行高度，其表达式为：

$$W(h) = 20 + 0.00003(h - 10000)^2. \quad (7)$$

5 问题一模型建立与求解

5.1 模型建立

5.1.1 升力模型

对于巡航阶段的固定翼飞机，其升力可由动压、机翼面积和升力系数共同表示为 [?]

$$L(t) = \frac{1}{2}\rho(h(t))V^2(t)SC_L(t), \quad (8)$$

其中， $\rho(h(t))$ 表示高度 $h(t)$ 处的大气密度， $V(t)$ 表示飞机空速， S 表示机翼面积， $C_L(t)$ 表示升力系数。

在小角度巡航爬升近似下，飞机垂直方向近似满足升力与重力平衡，即

$$L(t) = m(t)g. \quad (9)$$

由式 (4) 和式 (5) 可得升力系数为

$$C_L(t) = \frac{2m(t)g}{\rho(h(t))V^2(t)S}. \quad (10)$$

5.1.2 阻力模型

飞机巡航过程中的总阻力可表示为 [?]

$$D(t) = \frac{1}{2}\rho(h(t))V^2(t)SC_D(t), \quad (11)$$

其中， $C_D(t)$ 表示总阻力系数。本文采用经典抛物线阻力极曲线描述阻力系数 [?]，即

$$C_D(t) = C_{D0} + kC_L^2(t), \quad (12)$$

其中， C_{D0} 表示零升阻力系数， k 表示升致阻力因子。 C_L 表示升力系数

将式 (12) 代入式 (11)，可将总阻力分解为零升阻力和升致阻力：

$$D(t) = D_0(t) + D_i(t), \quad (13)$$

其中

$$D_0(t) = \frac{1}{2}\rho(h(t))V^2(t)SC_{D0}, \quad D_i(t) = \frac{1}{2}\rho(h(t))V^2(t)SkC_L^2(t). \quad (14)$$

零升阻力主要与机体外形、飞行速度和大气密度有关；升致阻力则来源于飞机产生升力的过程，其大小与升力系数的平方成正比。[?]

5.1.3 推力需求模型

在飞机航迹方向上，根据牛顿第二定律，可得到

$$m(t)\dot{V}(t) = F(t) - D(t) - m(t)g \sin \gamma, \quad (15)$$

其中， $\dot{V}(t)$ 表示空速变化率， $F(t)$ 表示发动机推力， $D(t)$ 表示巡航阻力， γ 表示航迹角。

高度变化率与航迹角之间满足

$$\dot{h}(t) = V(t) \sin \gamma. \quad (16)$$

将式 (16) 代入式 (15)，并整理可得发动机推力需求模型：

$$F(t) = D(t) + m(t)\dot{V}(t) + \frac{m(t)g}{V(t)}\dot{h}(t). \quad (17)$$

5.1.4 燃油消耗与质量变化模型

飞机巡航过程中，发动机推力输出会持续消耗燃油。根据推力燃油消耗率模型，单位时间燃油消耗率可表示为

$$\dot{m}_f(t) = c_f F(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right], \quad (18)$$

其中， c_f 表示推力燃油消耗系数， V_{opt} 表示参考最佳经济巡航速度， β 表示速度偏离惩罚系数。

由于巡航过程中除燃油外的其他载荷可近似认为保持不变，飞机总质量变化主要由燃油消耗引起，因此质量变化方程为

$$\dot{m}(t) = -\dot{m}_f(t). \quad (19)$$

将式 (18) 带入质量变化方程，可得

$$\dot{m}(t) = -c_f F(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right]. \quad (20)$$

5.1.5 水平运动方程

风场只改变飞机相对于地面的水平速度，不改变飞机相对于空气的空速。在小角度巡航近似下，飞机水平地速可表示为空速与高度相关风速的叠加，因此水平运动方程为 [?]

$$\dot{x}(t) = V(t) + W(h(t)), \quad (21)$$

其中， $W(h(t))$ 表示高度 $h(t)$ 处的水平风速。

5.1.6 综合状态方程组

综上，飞机巡航过程可描述为由飞行距离和飞机质量构成的状态方程组：

$$\begin{cases} \dot{x}(t) = V(t) + W(h(t)), \\ \dot{m}(t) = -c_f F(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right], \end{cases} \quad (22)$$

5.1.7 等速巡航爬升与等马赫数巡航爬升的高度演化规律

在巡航爬升过程中，飞机通常希望保持在较优气动效率附近飞行。为此，本文假设飞机在两种基准巡航爬升策略下均保持参考经济升力系数 $C_{L,c}$ 不变。该参考升力系数由初始状态确定：

$$C_{L,c} = \frac{2m_0 g}{\rho(h_0) V_0^2 S}. \quad (23)$$

根据升力平衡条件，有

$$\frac{1}{2} \rho(h(t)) V^2(t) S C_{L,c} = m(t) g. \quad (24)$$

由上式可知，在保持升力系数近似不变的条件下，飞机质量、飞行高度和空速之间存在确定的代数关系。随着燃油消耗导致飞机质量下降，飞机需要通过调整高度或速度来维持升力平衡。

等速巡航爬升策略 等速巡航爬升策略是指飞机在巡航爬升过程中保持空速不变，即

$$V(t) = V_c, \quad \dot{V}(t) = 0. \quad (25)$$

其中， V_c 可取初始空速 V_0 。将式 (25) 代入升力平衡式 (28)，可得

$$\rho(h(t)) = \frac{2m(t)g}{V_c^2 S C_{L,c}}. \quad (26)$$

其中 $\rho(h(t))$ 可由指数大气模型表示

$$\rho(h) = \rho_0 e^{-h/H_\rho}, \quad (27)$$

将两式联立则有

$$\rho_0 e^{-h(t)/H_\rho} = \frac{2m(t)g}{V_c^2 S C_{L,c}}. \quad (28)$$

由初始时刻同样满足升力平衡，可将式 (28) 写成相对形式：

$$\frac{m(t)}{m_0} = e^{-\frac{h(t)-h_0}{H_\rho}}. \quad (29)$$

于是，等速巡航爬升策略下高度关于质量的演化规律为

$$h(t) = h_0 - H_\rho \ln \frac{m(t)}{m_0}. \quad (30)$$

对式 (30) 关于时间求导，得到等速巡航爬升率：

$$\dot{h}(t) = -H_\rho \frac{\dot{m}(t)}{m(t)}. \quad (31)$$

由于飞机质量变化方程为

$$\dot{m}(t) = -\dot{m}_f(t), \quad (32)$$

因此

$$\dot{h}(t) = H_\rho \frac{\dot{m}_f(t)}{m(t)}. \quad (33)$$

上述式子表明，在等速巡航爬升策略下，飞机爬升率与单位质量燃油消耗率成正比。随着燃油消耗，飞机质量逐渐减小，为保持相同空速和相同升力系数，飞机需要进入密度更低的高空区域，因此高度随时间逐渐增加。

进一步地，在等速条件下，推力需求模型可化简为

$$F_c(t) = D_c(t) + \frac{m(t)g}{V_c} \dot{h}(t), \quad (34)$$

燃油消耗率为

$$\dot{m}_f(t) = c_f F_c(t) \left[1 + \beta (V_c - V_{\text{opt}})^2 \right]. \quad (35)$$

等马赫数巡航爬升策略 等马赫数巡航爬升策略指飞机在巡航爬升过程中保持马赫数不变，即：

$$M(t) = M_c, \quad V(t) = M_c a(h(t)). \quad (36)$$

其中， M_c 可由初始状态确定：

$$M_c = \frac{V_0}{a(h_0)}. \quad (37)$$

其中 $a(h_0)$ 表示初始高度的声速。

将等马赫数条件 (36) 代入升力平衡式 (24), 可得

$$\frac{1}{2}\rho(h(t))M_c^2 a^2(h(t))SC_{L,c} = m(t)g. \quad (38)$$

结合模型准备中的大气密度和声速模型, 可得

$$m(t) = \frac{SC_{L,c}M_c^2}{2g}\rho_0 e^{-\frac{h(t)}{H_\rho}} \gamma R_{\text{gas}} T(h(t)). \quad (39)$$

由初始时刻的升力平衡关系, 可将式 (37) 写成相对形式:

$$\frac{m(t)}{m_0} = e^{-\frac{h(t)-h_0}{H_\rho}} \frac{T(h(t))}{T(h_0)}. \quad (40)$$

为了得到爬升率表达式, 对式 (40) 两边取对数:

$$\ln \frac{m(t)}{m_0} = -\frac{h(t)-h_0}{H_\rho} + \ln \frac{T(h(t))}{T(h_0)}. \quad (41)$$

对时间求导, 得到

$$\frac{\dot{m}(t)}{m(t)} = -\frac{1}{H_\rho} \dot{h}(t) + \frac{T'(h(t))}{T(h(t))} \dot{h}(t). \quad (42)$$

由于 $T'(h) = -L_T$, 因此

$$\frac{\dot{m}(t)}{m(t)} = -\left[\frac{1}{H_\rho} + \frac{L_T}{T(h(t))} \right] \dot{h}(t). \quad (43)$$

结合质量变化方程 $\dot{m}(t) = -\dot{m}_f(t)$, 可得等马赫数巡航爬升率:

$$\dot{h}(t) = \frac{\dot{m}_f(t)}{m(t)} \left[\frac{1}{H_\rho} + \frac{L_T}{T(h(t))} \right]^{-1}. \quad (44)$$

式 (44) 表明, 等马赫数巡航爬升率不仅与单位质量燃油消耗率有关, 还受到温度随高度变化的影响。这是等马赫数策略与等速策略之间的主要区别。

进一步地, 等马赫数策略下空速随高度变化, 因此

$$\dot{V}(t) = M_c a'(h(t)) \dot{h}(t) = V(t) \frac{a'(h(t))}{a(h(t))} \dot{h}(t). \quad (45)$$

由 $a(h) = \sqrt{\gamma R_{\text{gas}} T(h)}$ 可得

$$\frac{a'(h)}{a(h)} = \frac{1}{2} \frac{T'(h)}{T(h)} = -\frac{L_T}{2T(h)}. \quad (46)$$

因此, 等马赫数策略下的空速变化率为

$$\dot{V}(t) = -\frac{L_T}{2T(h(t))} V(t) \dot{h}(t). \quad (47)$$

将式 (47) 代入推力需求模型, 可得

$$F_M(t) = D_M(t) + m(t) \left[\frac{g}{V(t)} - \frac{L_T V(t)}{2T(h(t))} \right] \dot{h}(t). \quad (48)$$

相应的燃油消耗率为

$$\dot{m}_f(t) = c_f F_M(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right], \quad (49)$$

其中

$$V(t) = M_c a(h(t)). \quad (50)$$

由式 (46)、式 (48) 和式 (49) 联立, 即可得到等马赫数巡航爬升策略下 $h(t)$ 与 $m(t)$ 的耦合微分方程组。数值求解该方程组后, 可获得等马赫数策略下的高度剖面、速度剖面和质量剖面。

5.1.8 两种策略的统一表达

从升力平衡关系出发, 可以得到两种策略下高度演化规律的统一形式。由

$$m(t) \propto \rho(h(t)) V^2(t) \quad (51)$$

可知

$$\frac{\dot{m}(t)}{m(t)} = \left[\frac{\rho'(h(t))}{\rho(h(t))} + 2 \frac{V'(h(t))}{V(h(t))} \right] \dot{h}(t). \quad (52)$$

由于 $\dot{m}(t) = -\dot{m}_f(t)$, 因此

$$\dot{h}(t) = \frac{\dot{m}_f(t)}{m(t)} \left[-\frac{\rho'(h(t))}{\rho(h(t))} - 2 \frac{V'(h(t))}{V(h(t))} \right]^{-1}. \quad (53)$$

对于等速策略, $V'(h) = 0$, 且 $\rho'(h)/\rho(h) = -1/H_\rho$, 于是式 (53) 退化为

$$\dot{h}(t) = H_\rho \frac{\dot{m}_f(t)}{m(t)}. \quad (54)$$

对于等马赫数策略, $V(h) = M_c a(h)$, 并有

$$\frac{V'(h)}{V(h)} = \frac{a'(h)}{a(h)} = -\frac{L_T}{2T(h)}, \quad (55)$$

于是式 (55) 退化为

$$\dot{h}(t) = \frac{\dot{m}_f(t)}{m(t)} \left[\frac{1}{H_\rho} + \frac{L_T}{T(h(t))} \right]^{-1}. \quad (56)$$

因此, 两种巡航爬升策略的差异本质上来自速度约束方式不同: 等速策略中空速保持不变, 高度变化主要由大气密度衰减平衡质量减小; 等马赫数策略中空速随声速变化而变化, 高度演化同时受到大气密度和温度变化的共同影响。

5.2 模型求解

基于式 (30) — 式 (35) 的等速巡航爬升模型, 以及式 (40) — 式 (49) 的等马赫数巡航爬升模型, 本文采用数值积分方法求解两种策略下的飞行剖面。计算以飞机质量由 $m_0 = 72450$ kg 下降至 $m_f = 62000$ kg 为停止条件, 并记录高度、质量、爬升率、飞行时间、航程和燃油消耗量。

首先由式 (23) 计算初始参考升力系数, 得

$$C_{L,c} = 0.6585.$$

该值处于合理巡航升力系数范围内, 因此可作为两种基准策略的参考气动状态。

等速策略中, 空速保持 $V(t) = V_0$ 。计算时, 根据式 (30) 由当前质量直接得到高度, 再利用式 (34) 和式 (35) 计算推力需求与燃油消耗率, 并由质量变化方程进行时间积分。水平航程由式 (21) 累计, 其中本文采用修正风场:

$$W(h) = 20 + 3 \times 10^{-5}(h - 10000)^2.$$

该形式在初始高度 $h_0 = 9500$ m 处对应风速为 27.5 m/s, 风速数量级较为合理。等马赫数策略中, 马赫数保持 $M(t) = M_0$ 。由式 (37) 计算得

$$M_0 = 0.7957.$$

由于式 (40) 给出的是高度与质量之间的隐式关系, 本文在每一个质量节点上采用一维方程求根法求解高度, 再由式 (36)、式 (48) 和式 (49) 依次计算空速、推力需求和燃油消耗率。通过上述求解过程, 可以分别得到两种策略下的高度剖面 $h(t)$ 、质量剖面 $m(t)$ 、爬升率剖面 $\dot{h}(t)$ 和航程剖面 $x(t)$ 。

5.3 结果分析

根据数值求解结果, 两种巡航爬升策略的主要指标如下表所示。

表 2: 巡航爬升策略对比

策略	最终高度/m	总飞行时间/min	总燃油消耗/kg	平均爬升率/(m/s)
等速爬升	10637.1	12.7076	10450	1.49132
等马赫数爬升	10437.8	13.5265	10450	1.15553

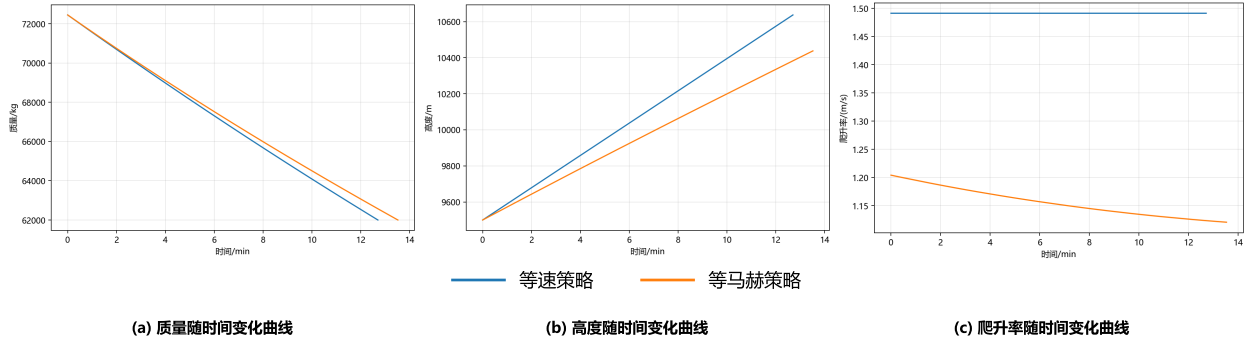


图 2: 飞行状态参数随时间变化对比示意图

由图 (a) 质量剖面可以看出, 两种策略均从初始质量 72450 kg 单调下降至终止质量 62000 kg。由于本问以终止质量作为积分停止条件, 因此两种策略的总燃油消耗量均为

$$m_0 - m_f = 72450 - 62000 = 10450 \text{ kg}.$$

由图 (b) 高度剖面可知, 两种策略均随燃油消耗逐渐爬升, 符合飞机减重后维持升力平衡的物理规律。等速巡航爬升策略的最终高度为 10637.1 m, 高于等马赫数巡航爬升策略的 10437.8 m。结合式 (30) 可知, 等速策略中空速保持不变, 质量降低后主要通过升高高度、降低空气密度来维持升力平衡; 而等马赫数策略由式 (40) 可知, 高度变化同时受到大气密度和温度变化影响, 因此最终高度较低。

由图 (c) 爬升率剖面可知, 等速策略的平均爬升率为 1.49132 m/s, 等马赫数策略的平均爬升率为 1.15553 m/s。在本文采用的参数和标准大气模型下, 等速策略爬升率更大。该结果可以由两种策略的爬升率表达式解释。等速策略下有式 (54)

$$\dot{h}_c(t) = H_\rho \frac{\dot{m}_f(t)}{m(t)}.$$

等马赫数策略下有式 (56)

$$\dot{h}_M(t) = \frac{\dot{m}_f(t)}{m(t)} \left[\frac{1}{H_\rho} + \frac{L_T}{T(h(t))} \right]^{-1}.$$

由于

$$\frac{1}{H_\rho} + \frac{L_T}{T(h(t))} > \frac{1}{H_\rho},$$

因此在单位质量燃油消耗率相近时, 等马赫数策略对应的爬升率系数小于等速策略。由此可见, 在本文模型下, 不能简单认为等马赫数巡航爬升策略一定具有更大的爬升率, 实际结论取决于标准大气模型、声速变化规律和速度约束方式。

由飞行时间可知,等速策略总飞行时间为 12.7076 min,等马赫数策略总飞行时间为 13.5265 min。等马赫数策略耗时更长,主要原因是其空速随声速降低而下降,平均空速低于等速策略。

综上,等速策略具有更高最终高度、更短飞行时间和更大平均爬升率;等马赫数策略飞行时间较长,最终航程略大。两种策略差异的本质来源是速度约束不同:等速策略主要通过密度变化平衡质量下降,等马赫数策略同时受到密度变化和声速变化影响。

6 问题二模型建立与求解

6.1 模型建立

6.1.1 巡航全过程燃油消耗积分模型

在问题一所建立的巡航动力学模型基础上,问题二进一步研究巡航全过程的燃油消耗量。飞机在巡航过程中所处的大气环境随高度连续变化,因此燃油消耗率不再是常数,而是高度、空速、质量以及大气参数共同作用下的非线性函数。

记单位时间燃油消耗率为

$$\dot{m}_f(t) = c_f F(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right], \quad (57)$$

其中推力需求 $F(t)$ 由问题一中的推力需求模型给出。于是,巡航全过程的总燃油消耗量可表示为:

$$J = \int_0^{t_f} \dot{m}_f(t) dt. \quad (58)$$

由于飞机水平运动方程为

$$\dot{x}(t) = V(t) + W(h(t)), \quad (59)$$

当 $V(t) + W(h(t)) > 0$ 时,可将时间积分转化为沿飞行路径的积分形式:

$$J = \int_0^{X_f} \frac{\dot{m}_f(x)}{V(x) + W(h(x))} dx. \quad (60)$$

式 (60) 表明,巡航总油耗不仅与单位时间燃油消耗率有关,还与飞机相对于地面的实际前进速度有关。若存在顺风,则相同空速下单位航程所需时间减少;若存在逆风,则单位航程所需时间增加,从而影响总燃油消耗。

6.1.2 燃油消耗率沿路径分布

为绘制燃油消耗率沿飞行路径的分布曲线,可将时间域上的数值解 $h(t), V(t), m(t), x(t)$ 转换为路径坐标下的函数。定义单位航程燃油消耗率为

$$q(x) = \frac{\dot{m}_f(x)}{V(x) + W(h(x))}. \quad (61)$$

则总燃油消耗量可写为

$$J = \int_0^{X_f} q(x) dx. \quad (62)$$

其中, $q(x)$ 表示飞机每飞行单位地面距离所消耗的燃油质量。相比于时间油耗率 $\dot{m}_f(t)$, 该指标更能反映巡航策略在实际航程上的经济性。

6.1.3 温度偏差修正模型

实际大气温度可能与标准大气存在偏差。设实际温度为

$$T_{\text{real}}(h) = T_{\text{std}}(h) + \Delta T(h), \quad (63)$$

其中, $\Delta T(h)$ 表示实际大气温度相对于标准大气温度的偏差。

当温度偏差较小时, 本文采用一阶泰勒展开进行近似修正。由声速表达式可得

$$a_{\Delta}(h) = \sqrt{\gamma R_{\text{gas}} [T_{\text{std}}(h) + \Delta T(h)]} = \sqrt{\gamma R_{\text{gas}} [T_{\text{std}}(h)]} \sqrt{1 + \frac{\Delta T(h)}{T_{\text{std}}(h)}} \approx a(h) \left[1 + \frac{\Delta T(h)}{2T_{\text{std}}(h)} \right]. \quad (64)$$

在局部压力近似不变的条件下, 由理想气体状态方程可得大气密度的一阶修正形式:

$$\rho_{\Delta}(h) \approx \rho(h) \left[1 - \frac{\Delta T(h)}{T_{\text{std}}(h)} \right]. \quad (65)$$

该近似表明, 在同一高度附近, 温度升高会导致密度降低, 从而改变升力、阻力以及推力需求。

将修正后的密度 $\rho_{\Delta}(h)$ 代入升力和阻力模型, 可得到修正后的阻力 $D_{\Delta}(t)$ 和推力需求 $F_{\Delta}(t)$ 。因此, 温度偏差修正后的燃油消耗率为

$$\dot{m}_{f,\Delta}(t) = c_f F_{\Delta}(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right]. \quad (66)$$

若采用等马赫数策略, 则空速还需由修正后的声速确定, 即

$$V_{\Delta}(t) = M_0 a_{\Delta}(h(t)). \quad (67)$$

最终, 修正后的总燃油消耗量为

$$J_{\Delta} = \int_0^{t_f} \dot{m}_{f,\Delta}(t) dt = \int_0^{X_f} \frac{\dot{m}_{f,\Delta}(x)}{V(x) + W(h(x))} dx. \quad (68)$$

通过比较 J_{Δ} 与标准大气条件下的 J , 即可分析温度偏差对巡航油耗计算结果的影响。当 $|\Delta T(h)|/T_{\text{std}}(h)$ 较小, 且其引起的油耗变化远小于工程允许误差时, 可安全忽略温度偏差; 反之, 则应采用修正后的大气参数重新计算油耗。

6.2 模型求解

基于路径积分模型，本文选取问题一中的等速巡航爬升轨迹作为标准大气下的基准轨迹。该轨迹包含 $h(t)$ 、 $V(t)$ 、 $m(t)$ 、 $\dot{m}_f(t)$ 和 $x(t)$ 等变量。将时间域结果转换为路径坐标后，逐点计算单位航程燃油消耗率 $q(x)$ ，并采用梯形积分法计算巡航全过程总油耗。具体求解步骤如下。

首先，读取问题一得到的等速巡航剖面。对每一个离散节点 i ，已知飞机高度 h_i 、空速 V_i 、质量 m_i 、燃油消耗率 $\dot{m}_{f,i}$ 和累计航程 x_i 。由水平运动方程计算地速：

$$U_i = V_i + W(h_i).$$

随后，根据式 (61) 中的单位航程油耗率定义，计算

$$q_i = \frac{\dot{m}_{f,i}}{U_i}.$$

最后，采用梯形积分得到标准大气条件下的累计燃油消耗量：

$$J(x_j) \approx \sum_{i=0}^{j-1} \frac{q_i + q_{i+1}}{2} (x_{i+1} - x_i).$$

当 $x_j = X_f$ 时，即得到该巡航策略在标准大气条件下的全过程总油耗 J 。

在大气参数处理上，等速策略主要体现密度 $\rho(h)$ 和风场 $W(h)$ 对路径油耗的影响；等马赫数策略中速度满足 $V = Ma(h)$ ，因此进一步体现了声速随高度变化对飞行状态和路径油耗计算的影响。两种轨迹共同用于验证大气参数高度分布对油耗积分结果的影响。

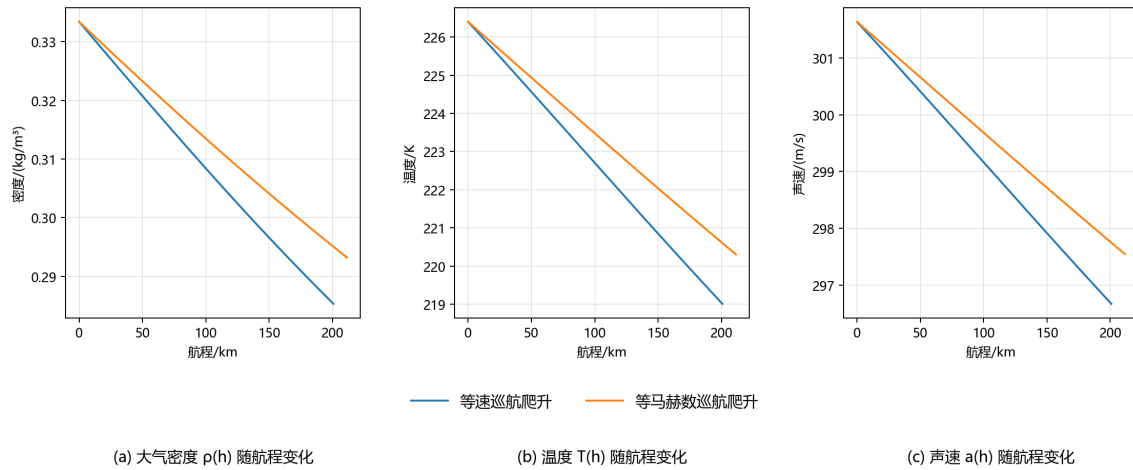


图 3: 温度、密度和声速与航程关系图

由图 3 可知,随着巡航爬升过程推进,大气密度、温度和声速均沿航程连续下降。等速策略最终高度更高,因此对应的大气密度、温度和声速下降幅度更大;等马赫数策略高度增长较缓,三类大气参数变化幅度相对较小。该结果说明飞机并非在单一大气状态下飞行,而是在随高度连续变化的参数场中完成路径积分。

针对实际大气温度偏差,本文采用一阶扰动近似进行修正。并分别设置恒定温度偏差 $\Delta T = 5K$, 和线性温差 $\Delta T(h) = 5 + 0.0005(h - h_0)$ 。在每个路径节点上,利用式 (64) 和式 (65) 更新声速和密度,再重新计算阻力、推力、燃油消耗率和单位航程油耗率,最终得到修正后的总燃油消耗量 J_{Δ} 。

6.3 结果分析

两种巡航策略在标准大气及温度偏差修正条件下的总油耗结果如下表所示。

表 3: 不同巡航策略与大气条件下的油耗对比

策略	情形	总油耗/kg	油耗变化/kg	平均单位航程油耗/(kg/m)
等速爬升	标准大气	10450	0	0.052039
等速爬升	恒定 5K 温差	10439.5	-10.4968	0.051987
等速爬升	线性温差	10439.1	-10.9384	0.051985
等马赫数爬升	标准大气	10450	0	0.049414
等马赫数爬升	恒定 5K 温差	10439.3	-10.7486	0.049363
等马赫数爬升	线性温差	10438.9	-11.1239	0.049362

由表可知,标准大气条件下,两种策略沿路径积分得到的总油耗均为 10450.0 kg,与由初始质量和终止质量计算得到的燃油消耗量一致。这说明路径积分模型与问题一中的质量变化模型相互吻合,能够正确描述巡航全过程的燃油消耗。为进一步分析油耗在飞行路径上的分布规律,图 4 给出了标准大气条件下两种策略的单位航程油耗率曲线。

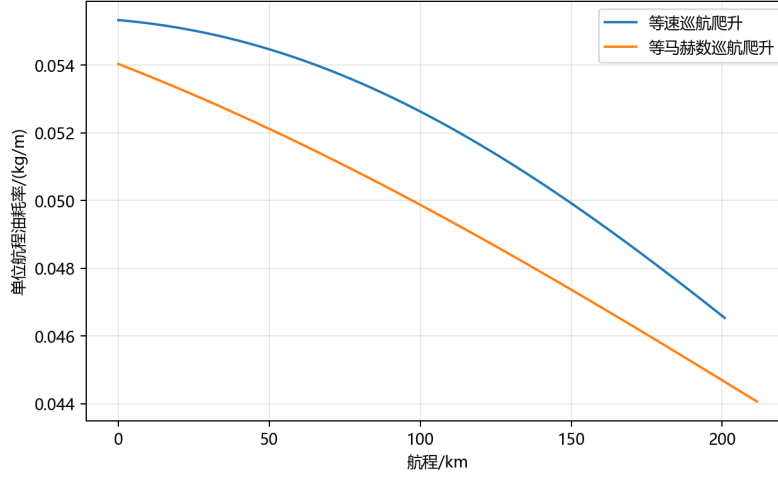
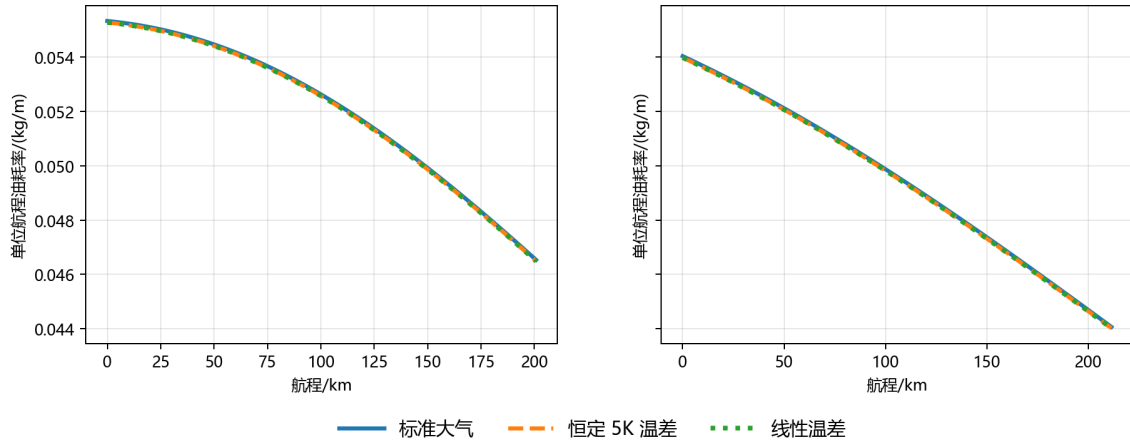


图 4: 标准大气条件下单位航程油耗率沿路径分布

从单位航程油耗率看，等速策略的平均单位航程油耗为 0.052039 kg/m ，高于等马赫数策略的 0.049414 kg/m 。这是因为等马赫数策略飞行时间更长、累计航程更大，在相同总燃油消耗下，其平均单位航程油耗更低。进一步从图 4 可知，单位航程油耗率并非常数，而是随质量下降、高度变化、阻力变化和地速变化连续改变，体现了高度分布参数场对路径油耗的累积作用。



(a) 等速巡航爬升

(b) 等马赫数巡航爬升

图 5: 温度偏差修正前后单位航程油耗率对比

为观察温度偏差对路径油耗分布的影响，图 5 分别给出了等速策略和等马赫数策略下温度修正前后的单位航程油耗率曲线。等速策略中，恒定 $5K$ 温差使总油耗减少 10.4968 kg ，线性温差使总油耗减少 10.9384 kg ，相对变化约为 -0.10% 。等马赫数策略中，恒定温差和线性温差对应的总油耗变化分别为 -10.7486 kg 和 -11.1239 kg ，相对变化约为 -0.10% 。两种策略下的修正幅度接近，说明在当前温度偏差范围内，温度扰动对最终总油耗影响较弱。

恒定温差和线性温差结果差异较小，主要原因是本题巡航高度变化范围有限。在线性温差模型中，等速策略对应的温差约从 $5K$ 增至 $5.57K$ ，等马赫数策略约从 $5K$ 增至 $5.47K$ ，相较于恒定 $5K$ 只增加约 $0.5K$ 。在一阶扰动近似下，温度影响主要通过 $\Delta T/T_{std}$ 这一小量进入密度和声速修正，因此传递到阻力、推力和总油耗后的差异较小。

从系统结构看，温度修正会改变密度和声速，其中密度变化进一步影响升力系数、阻力、推力和燃油率，声速变化影响马赫数计算以及等马赫状态解释。因此，温度偏差会增加模型的非线性耦合关系，但在当前设定下，其量级较小，最终油耗变化仅约 0.1% 。因此，当 $|\Delta T|/T_{std}$ 较小、飞行高度范围有限，且总油耗相对变化低于工程允许误差时，可在宏观策略比较中忽略温度偏差；若处于强温度异常环境、长航程累计计算或高精度油耗核算场景，则应保留温度修正项。

7 问题三模型建立与求解

7.1 模型建立

7.1.1 最优巡航控制模型

问题三要求在给定初始飞行状态下，确定使巡航总燃油消耗最小的高度-速度联合变化规律。由于飞机高度、速度、质量和阻力之间存在非线性耦合关系，本文将其建模为连续时间最优控制问题。

为避免推力模型中同时出现 $h(t)$ 、 $V(t)$ 及其导数造成表达不便，本文将飞行高度和空速也纳入状态变量，并引入高度变化率和空速变化率作为控制变量：

$$u_h(t) = \dot{h}(t), \quad u_v(t) = \dot{V}(t). \quad (69)$$

于是，状态变量选为

$$\mathbf{y}(t) = [x(t), m(t), h(t), V(t)]^T, \quad (70)$$

控制变量选为

$$\mathbf{u}(t) = [u_h(t), u_v(t)]^T. \quad (71)$$

在此表示下, 推力需求模型可写为

$$F(t) = D(t) + m(t)u_V(t) + \frac{m(t)g}{V(t)}u_h(t). \quad (72)$$

燃油消耗率为

$$\phi(t) = c_f F(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right]. \quad (73)$$

其中, $\phi(t)$ 即为单位时间燃油消耗率。

因此, 巡航过程的状态方程组可写为

$$\begin{cases} \dot{x}(t) = V(t) + W(h(t)), \\ \dot{m}(t) = -\phi(t), \\ \dot{h}(t) = u_h(t), \\ \dot{V}(t) = u_V(t). \end{cases} \quad (74)$$

若给定巡航航程 X_f , 则总燃油消耗最小问题可表示为

$$\min_{\mathbf{u}(t), t_f} J = \int_0^{t_f} \phi(t) dt. \quad (75)$$

对应的初始条件和终止条件为

$$x(0) = 0, \quad m(0) = m_0, \quad h(0) = h_0, \quad V(0) = V_0, \quad (76)$$

$$x(t_f) = X_f, \quad m(t_f) \geq m_f. \quad (77)$$

其中, X_f 表示给定巡航航程, m_f 表示终止时允许的最小剩余质量。

7.1.2 约束条件

飞机巡航轨迹应满足飞行包线、升力平衡、马赫数以及推力非负等约束。首先, 高度和速度需满足

$$h_{\min} \leq h(t) \leq h_{\max}, \quad V_{\min} \leq V(t) \leq V_{\max}. \quad (78)$$

马赫数需满足跨音速限制:

$$M(t) = \frac{V(t)}{a(h(t))} \leq M_{\max}. \quad (79)$$

升力系数由升力平衡确定, 为

$$C_L(t) = \frac{2m(t)g}{\rho(h(t))V^2(t)S}. \quad (80)$$

为保证飞机处于合理升力工作范围，设置

$$C_{L,\min} \leq C_L(t) \leq C_{L,\max}. \quad (81)$$

考虑到乘坐品质和结构安全，还需对爬升率和水平加速度进行限制：

$$|\dot{h}(t)| \leq \dot{h}_{\max}, \quad |\dot{V}(t)| \leq \dot{V}_{\max}, \quad (82)$$

其中 $\dot{h}_{\max} = 15 \text{ m/s}$, $\dot{V}_{\max} = 1.0 \text{ m/s}^2$

此外，发动机推力应满足

$$0 \leq F(t) \leq F_{\max}(h(t), V(t)). \quad (83)$$

若题目未给出最大推力包线，则可在基础模型中仅保留 $F(t) \geq 0$ ，并在模型改进中进一步引入发动机推力上限。

7.1.3 最优性必要条件

为分析最优轨迹满足的数学条件，引入伴随变量

$$\lambda_x(t), \lambda_m(t), \lambda_h(t), \lambda_V(t),$$

构造哈密顿函数

$$\begin{aligned} \mathcal{H} = & \phi(t) + \lambda_x(t) [V(t) + W(h(t))] - \lambda_m(t)\phi(t) \\ & + \lambda_h(t)u_h(t) + \lambda_V(t)u_V(t). \end{aligned} \quad (84)$$

整理可得

$$\mathcal{H} = [1 - \lambda_m(t)] \phi(t) + \lambda_x(t) [V(t) + W(h(t))] + \lambda_h(t)u_h(t) + \lambda_V(t)u_V(t). \quad (85)$$

对于内部最优解，其伴随方程满足

$$\dot{\lambda}_i(t) = -\frac{\partial \mathcal{H}}{\partial y_i}, \quad y_i \in \{x, m, h, V\}. \quad (86)$$

即

$$\begin{cases} \dot{\lambda}_x(t) = -\frac{\partial \mathcal{H}}{\partial x}, \\ \dot{\lambda}_m(t) = -\frac{\partial \mathcal{H}}{\partial m}, \\ \dot{\lambda}_h(t) = -\frac{\partial \mathcal{H}}{\partial h}, \\ \dot{\lambda}_V(t) = -\frac{\partial \mathcal{H}}{\partial V}. \end{cases} \quad (87)$$

同时，控制变量需满足驻值条件

$$\frac{\partial \mathcal{H}}{\partial u_h} = 0, \quad \frac{\partial \mathcal{H}}{\partial u_V} = 0. \quad (88)$$

当高度、速度、升力系数或推力约束处于边界时，上述驻值条件需替换为包含不等式约束乘子的 KKT 条件。

7.2 参数取值与依据

本文参考同类机型性能数据 [?][?]、飞行包线理论 [?] 及乘坐品质标准 [?]，设定各约束参数如表 4 所示。

表 4: 约束参数取值

参数	符号	取值	依据
最低高度	$h_{\min}$	8000 m	安全高度与航线最低航路高度
最高高度	$h_{\max}$	12000 m	飞机升限及发动机推力限制
最小空速	$V_{\min}$	180 m/s	失速速度
最大空速	$V_{\max}$	280 m/s	结构限制与跨音速边界
最大马赫数	$M_{\max}$	0.85	阻力发散马赫数
最小升力系数	$C_{L,\min}$	0.1	迎角过低时操纵性问题
最大升力系数	$C_{L,\max}$	1.2	失速迎角对应升力系数
最大爬升率	$\dot{h}_{\max}$	15 m/s	乘坐品质与结构载荷
最大水平加速度	$\dot{V}_{\max}$	1.0 m/s ²	乘坐舒适度与发动机响应

7.3 模型求解

7.3.1 直接离散化求解模型

由于本问题的燃油消耗率、阻力模型和风场模型均具有明显非线性，采用间接法求解两点边值问题较为困难。因此，本文采用直接法进行数值求解。将时间区间 $[0, t_f]$ 划分为 N 个小区间，节点记为 $t_i = i\Delta t$, $i = 0, 1, \dots, N$ 。

采用梯形积分近似，可将目标函数离散为

$$J \approx \sum_{i=0}^{N-1} \frac{\Delta t}{2} [\phi_i + \phi_{i+1}]. \quad (89)$$

状态方程离散为

$$\mathbf{y}_{i+1} = \mathbf{y}_i + \frac{\Delta t}{2} [\mathbf{f}(\mathbf{y}_i, \mathbf{u}_i) + \mathbf{f}(\mathbf{y}_{i+1}, \mathbf{u}_{i+1})], \quad (90)$$

其中, $\mathbf{f}$ 表示式 (76) 右端函数。这样即可将连续最优控制问题转化为带非线性等式与不等式约束的非线性规划问题。

7.3.2 风场对最优解结构的影响

在无风条件下, $W(h) = 0$, 飞机地速等于空速, 水平运动方程不再显式依赖高度。此时可引入能量高度 [?]

$$E_{\text{tot}}(t) = h(t) + \frac{V^2(t)}{2g}. \quad (91)$$

其变化率为

$$\dot{E}_{\text{tot}}(t) = \dot{h}(t) + \frac{V(t)}{g} \dot{V}(t). \quad (92)$$

结合推力需求模型可知, 无风条件下高度变化和速度变化可通过能量高度统一描述, 从而在一定条件下将高度-速度二维控制问题简化为能量高度的一维决策问题。

然而, 当存在垂直风切变时, 风速 $W(h)$ 显式出现在水平运动方程中, 并通过路径积分中的 $V(t) + W(h(t))$ 影响单位航程燃油消耗。此时, 即使能量高度相同, 不同高度对应的风速也可能不同, 导致相同能量状态下的地速和单位航程油耗不同。因此, 风场破坏了无风条件下高度与速度之间的简化关系, 使最优轨迹必须同时考虑气动效率、速度偏离损失和风场收益。

7.4 结果分析

8 问题四模型建立与求解

8.1 模型建立

8.1.1 综合数值计算框架

问题四需要在前三问模型基础上, 建立完整的巡航策略综合数值计算模型。本文采用统一的状态变量

$$\mathbf{y}(t) = [x(t), m(t), h(t), V(t)]^T,$$

并采用问题三中的状态方程作为基本动力学约束。

8.1.2 等速巡航爬升策略模型

等速巡航爬升策略中，空速保持为常数

$$V(t) = V_c, \quad \dot{V}(t) = 0. \quad (93)$$

其中， V_c 可取初始空速 V_0 或参考经济巡航速度 V_{opt} 。

为使飞机在质量逐渐减小的过程中保持合理的气动工作状态，本文令升力系数保持在初始巡航设计值 $C_{L,c}$ 附近。根据升力平衡关系，有

$$C_{L,c} = \frac{2m(t)g}{\rho(h(t))V_c^2 S}. \quad (94)$$

若采用指数大气模型，则由式 (93) 可得到高度随质量变化的关系：

$$h(t) = -H_\rho \ln \left[\frac{2m(t)g}{\rho_0 V_c^2 S C_{L,c}} \right]. \quad (95)$$

该式表明，随着燃油消耗导致飞机质量下降，为保持相同升力系数和固定空速，飞机应逐渐爬升至更高高度。

等速策略下的推力需求为

$$F_{\text{cs}}(t) = D_{\text{cs}}(t) + \frac{m(t)g}{V_c} \dot{h}(t), \quad (96)$$

其中， $D_{\text{cs}}(t)$ 由对应高度、质量和速度下的阻力模型计算得到。燃油消耗率为

$$\phi_{\text{cs}}(t) = c_f F_{\text{cs}}(t) \left[1 + \beta (V_c - V_{\text{opt}})^2 \right]. \quad (97)$$

相应状态方程为

$$\begin{cases} \dot{x}(t) = V_c + W(h(t)), \\ \dot{m}(t) = -\phi_{\text{cs}}(t). \end{cases} \quad (98)$$

在初始条件 $x(0) = 0, m(0) = m_0, h(0) = h_0$ 和终止质量 $m(t_f) = m_f$ 下，数值积分式 (97) 即可得到等速策略下的高度、速度、质量和航程剖面。

8.1.3 风场条件下的最优高度-速度联合轨迹模型

考虑风场时，采用问题三建立的直接优化模型。若以固定终止质量 m_f 为条件，为了比较相同燃油消耗下的航程能力，可建立最大航程模型：

$$\max_{\mathbf{u}(t), t_f} x(t_f), \quad (99)$$

其约束为

$$\begin{cases} \dot{x}(t) = V(t) + W(h(t)), \\ \dot{m}(t) = -\phi(t), \\ \dot{h}(t) = u_h(t), \\ \dot{V}(t) = u_V(t), \end{cases} \quad (100)$$

并满足

$$x(0) = 0, \quad m(0) = m_0, \quad h(0) = h_0, \quad V(0) = V_0, \quad (101)$$

$$m(t_f) = m_f. \quad (102)$$

同时需满足高度、速度、马赫数、升力系数和推力约束。

若以等速策略得到的航程 X_{cs} 为固定航程，则可建立最小油耗模型：

$$\min_{\mathbf{u}(t), t_f} J_{\text{opt}} = \int_0^{t_f} \phi(t) dt, \quad (103)$$

其终止条件为

$$x(t_f) = X_{cs}. \quad (104)$$

通过上述两种等价比较方式，可以分别得到固定燃油条件下的航程提升和固定航程条件下的燃油节省。

8.1.4 策略对比指标

设等速策略的总燃油消耗量为 J_{cs} ，最优策略的总燃油消耗量为 J_{opt} ，则固定航程条件下的燃油节省比例定义为

$$\eta_J = \frac{J_{cs} - J_{\text{opt}}}{J_{cs}} \times 100\%. \quad (105)$$

设等速策略在终止质量 m_f 下的航程为 X_{cs} ，最优策略在相同终止质量下的航程为 X_{opt} ，则航程提升比例定义为

$$\eta_X = \frac{X_{\text{opt}} - X_{cs}}{X_{cs}} \times 100\%. \quad (106)$$

8.1.5 模型扩展一：温度偏差实时修正

在实际飞行中，飞机可通过气象预报或机载传感器获得实时温度偏差信息。将标准大气温度修正为

$$T_{\text{real}}(h, t) = T_{\text{std}}(h) + \Delta T(h, t). \quad (107)$$

由此更新声速和密度：

$$a_{\text{real}}(h, t) = \sqrt{\gamma R_{\text{gas}} T_{\text{real}}(h, t)}, \quad (108)$$

$$\rho_{\text{real}}(h, t) \approx \rho(h) \left[1 - \frac{\Delta T(h, t)}{T_{\text{std}}(h)} \right]. \quad (109)$$

将 ρ_{real} 和 a_{real} 分别代入阻力模型、升力约束和马赫数约束，即可得到温度实时修正后的巡航优化模型。该扩展提高了模型对实际大气环境的适应能力，但需要实时气象数据支持，并会增加数值求解过程中的参数更新成本。

8.1.6 模型扩展二：发动机安装损失修正

实际飞机发动机安装在机翼或机身上后，会受到进气畸变、短舱干扰和喷流干扰等影响，使有效推力低于理想台架推力。为描述该影响，引入发动机安装效率 $\eta_{\text{ins}}(h, V)$ ，满足

$$0 < \eta_{\text{ins}}(h, V) \leq 1. \quad (110)$$

若 $F_{\text{req}}(t)$ 为飞机完成当前飞行动作所需的有效推力，则发动机实际输出推力可表示为

$$F_{\text{cmd}}(t) = \frac{F_{\text{req}}(t)}{\eta_{\text{ins}}(h(t), V(t))}. \quad (111)$$

因此，考虑安装损失后的燃油消耗率修正为

$$\phi_{\text{ins}}(t) = c_f F_{\text{cmd}}(t) \left[1 + \beta (V(t) - V_{\text{opt}})^2 \right]. \quad (112)$$

同时，推力约束应修正为

$$F_{\text{req}}(t) \leq \eta_{\text{ins}}(h(t), V(t)) F_{\text{max}}(h(t), V(t)). \quad (113)$$

该扩展使模型更接近真实发动机工作状态，但需要额外的发动机安装效率或推力包线数据。

8.2 模型求解

8.3 结果分析

9 灵敏度分析

9.0.1 速度偏离惩罚系数敏感性分析

速度偏离惩罚系数 β 刻画了空速偏离最佳效率速度后燃油消耗增加的强度。为分析模型对该参数的敏感性，令

$$\beta_{\alpha} = \alpha \beta_0, \quad (114)$$

其中， β_0 为题目给定的标称值， α 为扰动系数，可取 $\alpha \in \{0.8, 0.9, 1.0, 1.1, 1.2\}$ 。

对于每一个 β_α ，重新求解最优巡航轨迹，得到对应的最优目标函数值 $J^*(\beta_\alpha)$ 、最优高度轨迹 $h^*(t; \beta_\alpha)$ 和最优速度轨迹 $V^*(t; \beta_\alpha)$ 。定义目标函数对 β 的相对敏感度为

$$S_J = \frac{J^*(\beta_0(1 + \varepsilon)) - J^*(\beta_0(1 - \varepsilon))}{2\varepsilon J^*(\beta_0)}, \quad (115)$$

其中， ε 为小扰动比例。类似地，可定义最终高度、平均速度和总航程等指标的敏感度，从而识别模型中对速度惩罚系数变化最敏感的环节。

参考文献

- [1] Delgado Muñoz, L., & Prats Menéndez, X. (2009). Fuel consumption assessment for speed variation concepts during the cruise phase. In Conference on Air Traffic Management Economics.
- [2] Atmosphere, U. S. (1976). US Government Printing Office: Washington. DC, USA, 1-241.
- [3] 王兵, 张博雯, & 彭瑛. (2025). 基于历史航迹数据的民用航空器极曲线估算方法. 交通运输工程学报, 25(4), 328-339.
- [4] Valenzuela Romero, A., & Rivas Rivas, D. (2014). Analysis of Wind-Shear Effects on Optimal Aircraft Cruise. International Conference on Research in Air Transportation (ICRAT).
- [5] Anderson Jr, J. D. (1991). Fundamentals of Aerodynamics, McGraw-Hill. New York.
- [6] 陈兴荣. (1988). 飞机颠簸的气象因素分析. 中国民航学院学报 (综合版), (3), 36-45.
- [7] 吴慰祖. (1989). 飞机全尺寸阻力的预计. 飞行力学, (3), 11-27.
<https://doi.org/10.13645/j.cnki.f.d.1989.03.002>
- [8] Tyrén, C. (1982). Magnetic anomalies as a reference for ground-speed and map-matching navigation. The Journal of navigation, 35(2), 242-254.
- [9] Filippone, A. (2012). Advanced aircraft flight performance (Vol. 34). Cambridge University Press.
- [10] Torenbeek, E. (2013). Unified Cruise Performance.
- [11] 吴奕铭.(2022). 运输类飞机商业飞行的性能分析规范及实例研究 (硕士学位论文, 中国民用航空飞行学院). 硕士 <https://doi.org/10.27722/d.cnki.gzgmh.2022.000286>.
- [12] 高浩.(1984). 改善乘坐品质纵向增稳器反馈系数的选择. 航空学报,(1),18-23.

附录

```
1      %%%问题一
2      from __future__ import annotations
3
4      import sys
5      from pathlib import Path
6
7      import matplotlib
8
9      matplotlib.use("Agg")
10     import matplotlib.pyplot as plt
11     import numpy as np
12     import pandas as pd
13     from scipy.optimize import brentq
14
15     ROOT = Path(__file__).resolve().parents[1]
16     if str(ROOT) not in sys.path:
17         sys.path.insert(0, str(ROOT))
18
19     from common.aerodynamics import drag, lift_coefficient
20     from common.atmosphere import T_std, mach_number, sound_speed, wind_for_mode
21     from common.fuel_model import speed_penalty
22     from common.parameters import PARAMS, Parameters
23     from common.utils import (
24         cumulative_trapezoid_from_values,
25         ensure_dir,
26         save_dataframe,
27         save_figure,
28     )
29
```

```

30
31 def compute_constant_speed_profile(
32     n_points: int = 1200,
33     wind_mode: str = "strict",
34     params: Parameters = PARAMS,
35 ) -> pd.DataFrame:
36     """Constant-air-speed cruise-climb profile integrated over mass."""
37     mass = np.linspace(params.m0, params.mf, n_points)
38     fuel_used = params.m0 - mass
39     V = np.full_like(mass, params.V0)
40
41     # Constant CL gives  $h = h_0 - H_{rho} \ln(m/m_0)$ .
42     CL_c = lift_coefficient(params.m0, params.h0, params.V0, params)
43     h = params.h0 - params.Hrho * np.log(mass / params.m0)
44
45     D = drag(mass, h, V, params)
46     penalty = speed_penalty(V, params)
47     p = params.cF * penalty
48     coupling = params.Hrho * params.g / params.V0
49     denom = 1.0 - p * coupling
50     if np.any(denom <= 0):
51         raise RuntimeError("Constant-speed fuel-rate coupling denominator is non-positive.")
52     rate = p * D / denom
53     hdot = params.Hrho * rate / mass
54     thrust = rate / p
55     CL = lift_coefficient(mass, h, V, params)
56     mach = mach_number(V, h, params)
57     W = wind_for_mode(h, wind_mode)
58     ground_speed = V + W
59

```

```

60     time_s = cumulative_trapezoid_from_values(1.0 / rate, fuel_used)
61     distance_m = cumulative_trapezoid_from_values(ground_speed / rate, fuel_used)
62
63     return pd.DataFrame(
64         {
65             "time_s": time_s,
66             "height_m": h,
67             "mass_kg": mass,
68             "velocity_mps": V,
69             "mach": mach,
70             "lift_coefficient": CL,
71             "reference_lift_coefficient": CL_c,
72             "drag_N": D,
73             "thrust_N": thrust,
74             "fuel_rate_kgps": rate,
75             "climb_rate_mps": hdot,
76             "wind_mps": W,
77             "ground_speed_mps": ground_speed,
78             "distance_m": distance_m,
79             "fuel_used_kg": fuel_used,
80         }
81     )
82
83
84 def __constant_mach_height_from_mass(mass: float, params: Parameters) -> float:
85     T_ref = T_std(params.h0, params)
86     target = mass / params.m0
87
88     def relation(h):
89         return np.exp(-(h - params.h0) / params.Hrho) * T_std(h, params) / T_ref - target

```

```

90
91     if abs(mass - params.m0) < 1e-10:
92         return params.h0
93     return brentq(relation, params.h0, 20000.0, xtol=1e-10, rtol=1e-10)
94
95
96 def compute_constant_mach_profile(
97     n_points: int = 1200,
98     wind_mode: str = "strict",
99     params: Parameters = PARAMS,
100 ) -> pd.DataFrame:
101     """Constant-Mach cruise-climb profile integrated over mass."""
102     mass = np.linspace(params.m0, params.mf, n_points)
103     fuel_used = params.m0 - mass
104     M0 = params.V0 / sound_speed(params.h0, params)
105     h = np.array([_constant_mach_height_from_mass(m, params) for m in mass])
106     T = T_std(h, params)
107     V = M0 * sound_speed(h, params)
108
109     D = drag(mass, h, V, params)
110     penalty = speed_penalty(V, params)
111     p = params.cF * penalty
112
113     # Differentiate  $\ln(m/m0) = -(h-h0)/Hrho + \ln(T/T0h)$ .
114     A = -1.0 / params.Hrho - params.LT / T
115     B = -1.0 / (mass * A) # hdot = B*fuel_rate
116     dVdh = -V * params.LT / (2.0 * T)
117     denom = 1.0 - p * mass * B * (dVdh + params.g / V)
118     if np.any(denom <= 0):
119         raise RuntimeError("Constant-Mach fuel-rate coupling denominator is non-positive.")

```



```

120     rate = p * D / denom
121     hdot = B * rate
122     Vdot = dVdh * hdot
123     thrust = rate / p
124     CL = lift_coefficient(mass, h, V, params)
125     mach = mach_number(V, h, params)
126     W = wind_for_mode(h, wind_mode)
127     ground_speed = V + W
128
129     time_s = cumulative_trapezoid_from_values(1.0 / rate, fuel_used)
130     distance_m = cumulative_trapezoid_from_values(ground_speed / rate, fuel_used)
131
132     return pd.DataFrame(
133         {
134             "time_s": time_s,
135             "height_m": h,
136             "mass_kg": mass,
137             "velocity_mps": V,
138             "mach": mach,
139             "lift_coefficient": CL,
140             "drag_N": D,
141             "thrust_N": thrust,
142             "fuel_rate_kgps": rate,
143             "climb_rate_mps": hdot,
144             "acceleration_mps2": Vdot,
145             "wind_mps": W,
146             "ground_speed_mps": ground_speed,
147             "distance_m": distance_m,
148             "fuel_used_kg": fuel_used,
149         }

```

```

150     )
151
152
153 def build_summary(cs: pd.DataFrame, cm: pd.DataFrame, params: Parameters = PARAMS) -> pd.DataFrame:
154     rows = []
155     for strategy, df in [
156         ("constant_speed", cs),
157         ("constant_mach", cm),
158     ]:
159         total_time = float(df["time_s"].iloc[-1])
160         final_height = float(df["height_m"].iloc[-1])
161         rows.append(
162             {
163                 "strategy": strategy,
164                 "final_height_m": final_height,
165                 "total_time_s": total_time,
166                 "total_time_min": total_time / 60.0,
167                 "final_distance_m": float(df["distance_m"].iloc[-1]),
168                 "final_distance_km": float(df["distance_m"].iloc[-1]) / 1000.0,
169                 "fuel_consumption_kg": params.m0 - params.mf,
170                 "average_climb_rate_mps": (final_height - params.h0) / total_time,
171             }
172         )
173     return pd.DataFrame(rows)
174
175
176 def make_figures(cs: pd.DataFrame, cm: pd.DataFrame, figures_dir: Path):
177     ensure_dir(figures_dir)
178     plt.rcParams["font.sans-serif"] = [
179         "Microsoft YaHei",

```

```

180         "SimHei",
181         "Noto Sans CJK SC",
182         "Arial Unicode MS",
183         "DejaVu Sans",
184     ]
185     plt.rcParams["axes.unicode_minus"] = False
186     datasets = [("等速策略", cs), ("等马赫策略", cm)]
187
188     def line_plot(filename, y_col, y_label):
189         fig, ax = plt.subplots(figsize=(8, 5))
190         for label, df in datasets:
191             ax.plot(df["time_s"] / 60.0, df[y_col], label=label)
192             ax.set_xlabel("时间/min")
193             ax.set_ylabel(y_label)
194             ax.grid(True, alpha=0.3)
195             ax.legend()
196             save_figure(figures_dir / filename, fig)
197
198     line_plot("height_vs_time.png", "height_m", "高度/m")
199     line_plot("mass_vs_time.png", "mass_kg", "质量/kg")
200     line_plot("climb_rate_vs_time.png", "climb_rate_mps", "爬升率/(m/s)")
201
202
203     def write_explanation(summary: pd.DataFrame, path: Path):
204         cs = summary.loc[summary["strategy"] == "constant_speed"].iloc[0]
205         cm = summary.loc[summary["strategy"] == "constant_mach"].iloc[0]
206         content = f"""# 问题 1 数值说明
207
208     ## 求解目标
209

```

```

210
211 ## 数学模型
212
213 
$$F = D + m \dot{V} + m g \dot{h} / V$$

214
215 等马赫策略固定  $M_0 = V_0 / a(h_0)$ ，速度随高度由  $V(h) = M_0 a(h)$  给出，高度由隐式关系
216
217 
$$m/m_0 = \exp(-(h-h_0)/H_{rho}) T(h)/T(h_0)$$

218
219 用 scipy.optimize.brentq 逐点求解。
220
221 ## 输出文件
222
223 - problem1_constant_speed_profile.csv：等速策略完整剖面。
224 - problem1_constant_mach_profile.csv：等马赫数策略完整剖面。
225 - problem1_summary.csv：两种策略终值与总量对比。
226 - height_vs_time.png、mass_vs_time.png 和 climb_rate_vs_time.png：剖面图。
227
228 ## 关键结果
229
230 - 等速策略最终高度为 {cs['final_height_m']:.3f} m，总时间为 {cs['total_time_min']:.3f} min，
231 - 等马赫策略最终高度为 {cm['final_height_m']:.3f} m，总时间为 {cm['total_time_min']:.3f} min
232 - 两种策略燃油消耗均为从 'm0' 到 'mf' 的质量差，即 {cs['fuel_consumption_kg']:.3f} kg。
233
234 等速策略通过保持参考升力系数使高度随质量下降逐渐升高。等马赫数策略中声速随标准大气温度降低而
235
236 ## 风场数量级提醒
237
238 代码默认使用修正后的风场  $W(h) = 20 + 3 \cdot 10^{(-5)(h-10000)^2}$ 。在 9500 m 附近该式给出约 27.5 m/s 的
239 """

```

```

240     path.write_text(content, encoding="utf-8")
241     print(f"Saved explanation: {path}")
242
243
244 def main():
245     print("Problem 1: solving constant-speed and constant-Mach cruise climb.")
246     results_dir = ensure_dir(Path(__file__).resolve().parent / "results")
247     figures_dir = ensure_dir(Path(__file__).resolve().parent / "figures")
248
249     cs = compute_constant_speed_profile()
250     cm = compute_constant_mach_profile()
251     summary = build_summary(cs, cm)
252
253     save_dataframe(cs, results_dir / "problem1_constant_speed_profile.csv")
254     save_dataframe(cm, results_dir / "problem1_constant_mach_profile.csv")
255     save_dataframe(summary, results_dir / "problem1_summary.csv")
256     make_figures(cs, cm, figures_dir)
257     write_explanation(summary, Path(__file__).resolve().parent / "explanation.md")
258     print("Problem 1 complete.")
259
260
261 if __name__ == "__main__":
262     main()

```

```

1     """%%问题二
2
3     from __future__ import annotations
4
5     import sys
6     from pathlib import Path

```

```

7 import matplotlib
8
9 matplotlib.use("Agg")
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13
14 ROOT = Path(__file__).resolve().parents[1]
15 if str(ROOT) not in sys.path:
16     sys.path.insert(0, str(ROOT))
17
18 from common.aerodynamics import drag, lift_coefficient
19 from common.atmosphere import T_std, mach_number, rho_exp, sound_speed
20 from common.fuel_model import fuel_rate
21 from common.parameters import PARAMS, Parameters
22 from common.utils import (
23     cumulative_trapezoid_from_values,
24     ensure_dir,
25     relative_change,
26     save_dataframe,
27     save_figure,
28 )
29 from problem1.solve_problem1 import compute_constant_mach_profile, compute_constant_speed_p
30
31 plt.rcParams["font.sans-serif"] = [
32     "Microsoft YaHei",
33     "SimHei",
34     "SimSun",
35     "Arial Unicode MS",
36     "DejaVu Sans",

```

```

37 ]
38 plt.rcParams["axes.unicode_minus"] = False
39
40 STRATEGY_LABELS = {
41     "constant_speed": "等速巡航爬升",
42     "constant_mach": "等马赫数巡航爬升",
43 }
44 STRATEGY_ORDER = ["constant_speed", "constant_mach"]
45 CASE_LABELS = {
46     "standard": "标准大气",
47     "deltaT_constant_5K": "恒定 5K 温差",
48     "deltaT_linear": "线性温差",
49 }
50 CASE_ORDER = ["standard", "deltaT_constant_5K", "deltaT_linear"]
51 CASE_STYLES = {
52     "standard": {"color": "#1f77b4", "linestyle": "-", "linewidth": 2.6, "zorder": 3},
53     "deltaT_constant_5K": {
54         "color": "#ff7f0e",
55         "linestyle": "--",
56         "linewidth": 2.6,
57         "zorder": 4,
58     },
59     "deltaT_linear": {
60         "color": "#2ca02c",
61         "linestyle": ":",
62         "linewidth": 3.0,
63         "zorder": 5,
64     },
65 }
66

```

```

67
68 def __path_integral(q_kg_per_m: np.ndarray, distance_m: np.ndarray) -> np.ndarray:
69     return cumulative_trapezoid_from_values(q_kg_per_m, distance_m)
70
71
72 def standard_path_fuel(profile: pd.DataFrame, strategy: str) -> pd.DataFrame:
73     df = profile.copy()
74     df["path_fuel_kg_per_m"] = df["fuel_rate_kgps"] / df["ground_speed_mps"]
75     df["cumulative_path_fuel_kg"] = __path_integral(
76         df["path_fuel_kg_per_m"].to_numpy(), df["distance_m"].to_numpy()
77     )
78     df["strategy"] = strategy
79     df["case"] = "standard"
80     return df[
81         [
82             "strategy",
83             "case",
84             "distance_m",
85             "time_s",
86             "height_m",
87             "mass_kg",
88             "velocity_mps",
89             "wind_mps",
90             "ground_speed_mps",
91             "fuel_rate_kgps",
92             "path_fuel_kg_per_m",
93             "cumulative_path_fuel_kg",
94             "mach",
95             "lift_coefficient",
96         ]

```



```

97     ]
98
99
100 def _delta_t_constant(h: np.ndarray, params: Parameters) -> np.ndarray:
101     return np.full_like(h, 5.0, dtype=float)
102
103
104 def _delta_t_linear(h: np.ndarray, params: Parameters) -> np.ndarray:
105     return 5.0 + 0.0005 * (h - params.h0)
106
107
108 def temperature_corrected_path_fuel(
109     profile: pd.DataFrame,
110     delta_t_func,
111     case_name: str,
112     strategy: str,
113     params: Parameters = PARAMS,
114 ) -> pd.DataFrame:
115     h = profile["height_m"].to_numpy()
116     V = profile["velocity_mps"].to_numpy()
117     m = profile["mass_kg"].to_numpy()
118     hdot = profile["climb_rate_mps"].to_numpy()
119     Vdot = (
120         profile["acceleration_mps2"].to_numpy()
121         if "acceleration_mps2" in profile.columns
122         else np.zeros_like(h)
123     )
124     U = profile["ground_speed_mps"].to_numpy()
125     distance = profile["distance_m"].to_numpy()
126

```

```

127 T_base = T_std(h, params)
128 rho_base = rho_exp(h, params)
129 a_base = sound_speed(h, params)
130 delta_t = delta_t_func(h, params)
131
132 # First-order perturbation specified in the task statement.
133 a_delta = a_base * (1.0 + delta_t / (2.0 * T_base))
134 rho_delta = rho_base * (1.0 - delta_t / T_base)
135 rho_delta = np.maximum(rho_delta, 1e-6)
136
137 D_delta = drag(m, h, V, params, rho=rho_delta)
138 thrust_delta = D_delta + m * Vdot + m * params.g * hdot / V
139 rate_delta = fuel_rate(thrust_delta, V, params)
140 q_delta = rate_delta / U
141 cumulative = _path_integral(q_delta, distance)
142 CL_delta = lift_coefficient(m, h, V, params, rho=rho_delta)
143 mach_delta = V / a_delta
144
145 return pd.DataFrame(
146     {
147         "strategy": strategy,
148         "case": case_name,
149         "distance_m": distance,
150         "time_s": profile["time_s"].to_numpy(),
151         "height_m": h,
152         "mass_kg": m,
153         "velocity_mps": V,
154         "wind_mps": profile["wind_mps"].to_numpy(),
155         "ground_speed_mps": U,
156         "delta_T_K": delta_t,

```

```

157         "T_std_K": T_base,
158         "T_real_K": T_base + delta_t,
159         "rho_standard_kgpm3": rho_base,
160         "rho_corrected_kgpm3": rho_delta,
161         "sound_speed_standard_mps": a_base,
162         "sound_speed_corrected_mps": a_delta,
163         "drag_N": D_delta,
164         "thrust_N": thrust_delta,
165         "fuel_rate_kgps": rate_delta,
166         "path_fuel_kg_per_m": q_delta,
167         "cumulative_path_fuel_kg": cumulative,
168         "mach_corrected": mach_delta,
169         "lift_coefficient_corrected": CL_delta,
170     }
171 )
172
173
174 def build_summary(standard: pd.DataFrame, corrected: pd.DataFrame, strategy: str) -> pd.DataFrame:
175     rows = []
176     standard_total = float(standard["cumulative_path_fuel_kg"].iloc[-1])
177     for case_name, df in [("standard", standard), *list(corrected.groupby("case"))]:
178         total = float(df["cumulative_path_fuel_kg"].iloc[-1])
179         rows.append(
180             {
181                 "strategy": strategy,
182                 "case": case_name,
183                 "total_fuel_kg": total,
184                 "absolute_change_kg": total - standard_total,
185                 "relative_change_percent": 100.0 * relative_change(total, standard_total),
186                 "max_fuel_rate_kgps": float(df["fuel_rate_kgps"].max()),

```

```

187         "mean_path_fuel_kg_per_m": total / float(df["distance_m"].iloc[-1]),
188     }
189 )
190 return pd.DataFrame(rows)
191
192
193 def _figure_path(figures_dir: Path, stem: str, suffix: str) -> Path:
194     return figures_dir / f"{stem}{suffix}.png"
195
196
197 def _save_figure_to_paths(fig: plt.Figure, *paths: Path, dpi: int = 180) -> None:
198     for path in paths:
199         ensure_dir(path.parent)
200         fig.savefig(path, dpi=dpi, bbox_inches="tight")
201         print(f"Saved figure: {path}")
202     plt.close(fig)
203
204
205 def _caption(fig: plt.Figure, text: str, y: float = 0.035, fontsize: int = 13) -> None:
206     fig.text(0.5, y, text, ha="center", va="center", fontsize=fontsize)
207
208
209 def _subcaption(fig: plt.Figure, ax: plt.Axes, text: str, offset: float = 0.075) -> None:
210     box = ax.get_position()
211     fig.text(
212         0.5 * (box.x0 + box.x1),
213         box.y0 - offset,
214         text,
215         ha="center",
216         va="center",

```

```

217         fontsize=12,
218     )
219
220
221 def _strategy_df(data: pd.DataFrame, strategy: str) -> pd.DataFrame:
222     return data[data["strategy"] == strategy].sort_values("distance_m")
223
224
225 def _case_df(data: pd.DataFrame, strategy: str, case_name: str) -> pd.DataFrame:
226     return data[(data["strategy"] == strategy) & (data["case"] == case_name)].sort_values(
227         "distance_m"
228     )
229
230
231 def make_figures(standard: pd.DataFrame, corrected: pd.DataFrame, figures_dir: Path, suffix: str):
232     """Legacy single-strategy figures kept for the original folder contract."""
233     ensure_dir(figures_dir)
234     strategy = str(standard["strategy"].iloc[0])
235     strategy_label = STRATEGY_LABELS.get(strategy, strategy)
236
237     fig, ax = plt.subplots(figsize=(8, 5))
238     ax.plot(standard["distance_m"] / 1000.0, standard["path_fuel_kg_per_m"], label=strategy_label)
239     ax.set_xlabel("航程/km")
240     ax.set_ylabel("单位航程油耗率/(kg/m)")
241     ax.grid(True, alpha=0.3)
242     ax.legend()
243     fig.subplots_adjust(bottom=0.18)
244     _caption(fig, f"标准大气条件下{strategy_label}单位航程油耗率沿路径分布")
245     save_figure(_figure_path(figures_dir, "path_fuel_rate_standard", suffix), fig)
246

```

```

247 fig, ax = plt.subplots(figsize=(8, 5))
248 ax.plot(standard["distance_m"] / 1000.0, standard["path_fuel_kg_per_m"], label="标准大气")
249 for case in CASE_ORDER[1:]:
250     df = corrected[corrected["case"] == case]
251     ax.plot(df["distance_m"] / 1000.0, df["path_fuel_kg_per_m"], label=CASE_LABELS[case])
252 ax.set_xlabel("航程/km")
253 ax.set_ylabel("单位航程油耗率/(kg/m)")
254 ax.grid(True, alpha=0.3)
255 ax.legend()
256 fig.subplots_adjust(bottom=0.18)
257 _caption(fig, f"{strategy_label} 温度偏差修正前后单位航程油耗率对比")
258 save_figure(_figure_path(figures_dir, "path_fuel_rate_temperature_comparison", suffix),
259
260 fig, ax = plt.subplots(figsize=(8, 5))
261 ax.plot(
262     standard["distance_m"] / 1000.0,
263     standard["cumulative_path_fuel_kg"],
264     label="标准大气",
265 )
266 for case in CASE_ORDER[1:]:
267     df = corrected[corrected["case"] == case]
268     ax.plot(df["distance_m"] / 1000.0, df["cumulative_path_fuel_kg"], label=CASE_LABELS[case])
269 ax.set_xlabel("航程/km")
270 ax.set_ylabel("累计燃油消耗量/kg")
271 ax.grid(True, alpha=0.3)
272 ax.legend()
273 fig.subplots_adjust(bottom=0.18)
274 _caption(fig, f"{strategy_label} 累计燃油消耗量随航程变化曲线")
275 save_figure(_figure_path(figures_dir, "cumulative_fuel_vs_distance", suffix), fig)
276

```

```

277
278 def make_strategy_comparison_figures(
279     standard_all: pd.DataFrame,
280     corrected_all: pd.DataFrame,
281     summary_all: pd.DataFrame,
282     figures_dir: Path,
283 ):
284     ensure_dir(figures_dir)
285
286     fig, ax = plt.subplots(figsize=(8, 5))
287     for strategy in STRATEGY_ORDER:
288         df = _strategy_df(standard_all, strategy)
289         ax.plot(
290             df["distance_m"] / 1000.0,
291             df["path_fuel_kg_per_m"],
292             label=STRATEGY_LABELS[strategy],
293         )
294         ax.set_xlabel("航程/km")
295         ax.set_ylabel("单位航程油耗率/(kg/m)")
296         ax.grid(True, alpha=0.3)
297         ax.legend()
298     fig.subplots_adjust(bottom=0.14)
299     _save_figure_to_paths(
300         fig,
301         figures_dir / "figure_5_5_standard_path_fuel_rate.png",
302         figures_dir / "path_fuel_rate_standard_strategy_comparison.png",
303     )
304
305     fig, ax = plt.subplots(figsize=(8, 5))
306     for strategy in STRATEGY_ORDER:

```

```

307     df = _strategy_df(standard_all, strategy)
308     ax.plot(
309         df["distance_m"] / 1000.0,
310         df["cumulative_path_fuel_kg"],
311         label=STRATEGY_LABELS[strategy],
312     )
313     ax.set_xlabel("航程/km")
314     ax.set_ylabel("累计燃油消耗量/kg")
315     ax.grid(True, alpha=0.3)
316     ax.legend()
317     fig.subplots_adjust(bottom=0.14)
318     _save_figure_to_paths(
319         fig,
320         figures_dir / "figure_5_6_cumulative_fuel_vs_distance.png",
321         figures_dir / "cumulative_fuel_strategy_comparison.png",
322     )
323
324     fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
325     for ax, strategy, subcaption in zip(
326         axes,
327         STRATEGY_ORDER,
328         ["(a) 等速巡航爬升", "(b) 等马赫数巡航爬升"],
329     ):
330         standard = _strategy_df(standard_all, strategy)
331         ax.plot(
332             standard["distance_m"] / 1000.0,
333             standard["path_fuel_kg_per_m"],
334             label=CASE_LABELS["standard"],
335             **CASE_STYLES["standard"],
336         )

```



```

337     for case in CASE_ORDER[1:]:
338         df = _case_df(corrected_all, strategy, case)
339         ax.plot(
340             df["distance_m"] / 1000.0,
341             df["path_fuel_kg_per_m"],
342             label=CASE_LABELS[case],
343             **CASE_STYLES[case],
344         )
345         ax.set_xlabel("航程/km")
346         ax.set_ylabel("单位航程油耗率/(kg/m)")
347         ax.grid(True, alpha=0.3)
348 handles, labels = axes[0].get_legend_handles_labels()
349 fig.subplots_adjust(bottom=0.36, top=0.96, wspace=0.22)
350 fig.legend(
351     handles,
352     labels,
353     loc="lower center",
354     bbox_to_anchor=(0.5, 0.16),
355     ncol=3,
356     frameon=False,
357     fontsize=12,
358     handlelength=2.4,
359     columnspacing=1.8,
360 )
361 for ax, text in zip(axes, ["(a) 等速巡航爬升", "(b) 等马赫数巡航爬升"]):
362     _subcaption(fig, ax, text, offset=0.27)
363 _save_figure_to_paths(
364     fig,
365     figures_dir / "figure_5_7_temperature_path_fuel_comparison.png",
366     figures_dir / "path_fuel_rate_temperature_strategy_comparison.png",

```

```

367 )
368
369 delta_summary = summary_all[summary_all["case"].isin(CASE_ORDER[1:])]
370 x = np.arange(2)
371 width = 0.34
372 fig, ax = plt.subplots(figsize=(8, 5))
373 for offset, strategy in [(-0.5 * width, "constant_speed"), (0.5 * width, "constant_mach")]
374     values = [
375         float(
376             delta_summary[
377                 (delta_summary["strategy"] == strategy) & (delta_summary["case"] == case)
378             ]["relative_change_percent"].iloc[0]
379         )
380         for case in CASE_ORDER[1:]
381     ]
382 bars = ax.bar(x + offset, values, width=width, label=STRATEGY_LABELS[strategy])
383 for bar, value in zip(bars, values):
384     ax.text(
385         bar.get_x() + bar.get_width() / 2.0,
386         value * 0.55,
387         f"{value:.4f}%",
388         ha="center",
389         va="center",
390         fontsize=10,
391         color="white",
392     )
393 ax.axhline(0.0, color="black", linewidth=0.8)
394 ax.set_xticks(x)
395 ax.set_xticklabels([CASE_LABELS[case] for case in CASE_ORDER[1:]])
396 ax.set_ylabel("总油耗相对变化 / %")

```

```

397 ax.grid(True, axis="y", alpha=0.3)
398 ax.legend()
399 fig.subplots_adjust(bottom=0.14)
400 _save_figure_to_paths(fig, figures_dir / "figure_5_8_temperature_relative_change.png")
401
402 fig, axes = plt.subplots(1, 3, figsize=(15, 5.6))
403 atmosphere_specs = [
404     ("rho", "密度/(kg/m³)", "(a) 大气密度 (h) 随航程变化"),
405     ("temperature", "温度/K", "(b) 温度 T(h) 随航程变化"),
406     ("sound_speed", "声速/(m/s)", "(c) 声速 a(h) 随航程变化"),
407 ]
408 for strategy in STRATEGY_ORDER:
409     df = _strategy_df(standard_all, strategy)
410     h = df["height_m"].to_numpy()
411     x_km = df["distance_m"].to_numpy() / 1000.0
412     values = {
413         "rho": rho_exp(h),
414         "temperature": T_std(h),
415         "sound_speed": sound_speed(h),
416     }
417     for ax, (key, _, _) in zip(axes, atmosphere_specs):
418         ax.plot(x_km, values[key], label=STRATEGY_LABELS[strategy])
419 for ax, (_, ylabel, _) in zip(axes, atmosphere_specs):
420     ax.set_xlabel("航程/km")
421     ax.set_ylabel(ylabel)
422     ax.grid(True, alpha=0.3)
423 handles, labels = axes[0].get_legend_handles_labels()
424 fig.subplots_adjust(bottom=0.36, top=0.96, wspace=0.32)
425 fig.legend(
426     handles,

```

```

427         labels ,
428         loc="lower center",
429         bbox_to_anchor=(0.5, 0.155),
430         ncol=2,
431         frameon=False ,
432         fontsize=12,
433         handlelength=2.4,
434         columnspring=2.2,
435     )
436     for ax, (_, _, text) in zip(axes, atmosphere_specs):
437         _subcaption(fig, ax, text, offset=0.27)
438     _save_figure_to_paths(fig, figures_dir / "figure_5_9_atmosphere_parameters_along_path.png")
439
440     standard_summary = summary_all[summary_all["case"] == "standard"]
441     fig, ax = plt.subplots(figsize=(8, 5))
442     ax.bar(
443         [STRATEGY_LABELS.get(x, x) for x in standard_summary["strategy"]],
444         standard_summary["mean_path_fuel_kg_per_m"],
445         color=["#2563eb", "#f97316"],
446     )
447     ax.set_ylabel("平均单位航程油耗率/(kg/m)")
448     ax.grid(True, axis="y", alpha=0.3)
449     fig.subplots_adjust(bottom=0.18)
450     _caption(fig, "标准大气条件下平均单位航程油耗率对比")
451     save_figure(figures_dir / "mean_path_fuel_strategy_bar.png", fig)
452
453
454 def write_explanation(summary_all: pd.DataFrame, path: Path):
455     rows = []
456     for _, row in summary_all.iterrows():

```

```

457         rows.append(
458             f"– {row['strategy']} / {row['case']}: total_fuel={row['total_fuel_kg']:.3 f} kg,
459             f"change={row['absolute_change_kg']:.3 f} kg "
460             f"({row['relative_change_percent']:.4 f}%)."
461         )
462     result_text = "\n".join(rows)
463     standards = summary_all[summary_all["case"] == "standard"].set_index("strategy")
464     cs = standards.loc["constant_speed"]
465     cm = standards.loc["constant_mach"]
466     content = f"""# 问题 2 数值说明
467
468 ## 求解目标
469
470 本问分别将问题 1 的等速巡航爬升轨迹和等马赫数巡航爬升轨迹作为输入，建立沿飞行路径的单位航程油
471
472 ## 路径积分模型
473
474 地速取 ' $U(h,V)=V+W(h)$ '，单位航程油耗为：
475
476 ' $q(x) = \text{fuel\_rate} / U(h,V)$ '
477
478 总燃油消耗沿路径积分：
479
480 ' $J = \int q(x) dx$ '
481
482 这种写法直接衡量每飞行 1 m 地面航程需要消耗的燃油，比单纯时间积分更适合比较航程经济性。时间和
483
484 ## 大气参数进入方式
485
486 — 密度 ' $\rho(h)$ ' 进入升力系数 ' $CL$ ' 和阻力 ' $D$ '。

```

```

487 - 温度 'T(h)' 进入声速 'a=sqrt(gamma R T)'，进而影响马赫数和等马赫速度关系。
488 - 声速 'a(h)' 进入 'M=V/a(h)'，用于约束和物理解释。
489
490 温度偏差采用题目要求的一阶扰动近似：
491
492 'a_delta = a [1 + DeltaT/(2T_std)]'
493
494 'rho_delta = rho [1 - DeltaT/T_std]'
495
496 代码计算了两种偏差：'DeltaT=5 K' 和 'DeltaT(h)=5+0.0005(h-h0)'。
497
498 ## 输出文件
499
500 - 'problem2_path_fuel_standard.csv'：等速策略标准大气路径油耗曲线。
501 - 'problem2_path_fuel_temperature_corrected.csv'：等速策略两种温度修正曲线。
502 - 'problem2_temperature_summary.csv'：等速策略汇总。
503 - 'problem2_path_fuel_standard_constant_mach.csv'：等马赫策略标准大气路径油耗曲线。
504 - 'problem2_path_fuel_temperature_corrected_constant_mach.csv'：等马赫策略两种温度修正曲线。
505 - 'problem2_temperature_summary_constant_mach.csv'：等马赫策略汇总。
506 - 'problem2_path_fuel_standard_all_strategies.csv'、'problem2_path_fuel_temperature_corrected_all_strategies.csv'：
507   所有策略标准大气和温度修正后的油耗曲线。
508 - 'figure_5_5_standard_path_fuel_rate.png'：图 5-5，标准大气条件下单位航程油耗率沿路径分布。
509 - 'figure_5_6_cumulative_fuel_vs_distance.png'：图 5-6，标准大气条件下累计燃油消耗量随航程变化。
510 - 'figure_5_7_temperature_path_fuel_comparison.png'：图 5-7，温度偏差修正前后单位航程油耗率对比。
511 - 'figure_5_8_temperature_relative_change.png'：图 5-8，温度偏差修正下总油耗相对变化。
512 - 'figure_5_9_atmosphere_parameters_along_path.png'：图 5-9，标准大气参数沿飞行路径变化曲线。
513
514 ## 图像说明
515
516 图 5-5 给出了标准大气条件下两种策略的单位航程油耗率沿路径分布。等速策略单位航程油耗率整体高于

```

```

517 图 5-6 用累计燃油消耗量验证路径积分模型。两条曲线最终均达到 ‘m0-mf=10450 kg’，说明将时间油耗
518
519 图 5-7 分别比较等速巡航爬升和等马赫数巡航爬升在温度修正前后的单位航程油耗率。温度修正后曲线整
520
521 图 5-8 使用相对变化而非总油耗绝对值展示温度修正影响。两种策略下温度修正引起的总油耗变化均约为
522
523 图 5-9 展示标准大气参数沿飞行路径的连续变化。随着飞机爬升，高度不断变化，大气密度、温度和声速
524
525 ## 关键结果
526
527 标准大气下，等速策略路径积分油耗为 {cs['total_fuel_kg']:.3f} kg，平均单位航程油耗为 {cs['mea
528
529 {result_text}
530
531 温度升高会降低一阶近似密度，通常会改变阻力、所需推力和燃油率；同时声速提高会降低同一空速下的
532
533 ## 风场说明
534
535 本问默认沿用修正后的风场 ‘ $W(h)=20+3*10^{(-5)}(h-10000)^2$ ’。在问题 1 等速轨迹高度范围内，风速约
536 """
537     path.write_text(content, encoding="utf-8")
538     print(f"Saved explanation: {path}")
539
540
541 def main():
542     print("Problem 2: computing path fuel integral and temperature corrections.")
543     base_dir = Path(__file__).resolve().parent
544     results_dir = ensure_dir(base_dir / "results")
545     figures_dir = ensure_dir(base_dir / "figures")
546

```

```

547 profile_speed = compute_constant_speed_profile()
548 profile_mach = compute_constant_mach_profile()
549
550 standard_speed = standard_path_fuel(profile_speed , "constant_speed")
551 corrected_speed_cases = [
552     temperature_corrected_path_fuel(
553         profile_speed , _delta_t_constant , "deltaT_constant_5K" , "constant_speed"
554     ) ,
555     temperature_corrected_path_fuel(
556         profile_speed , _delta_t_linear , "deltaT_linear" , "constant_speed"
557     ) ,
558 ]
559 corrected_speed = pd.concat(corrected_speed_cases , ignore_index=True)
560 summary_speed = build_summary(standard_speed , corrected_speed , "constant_speed")
561
562 standard_mach = standard_path_fuel(profile_mach , "constant_mach")
563 corrected_mach_cases = [
564     temperature_corrected_path_fuel(
565         profile_mach , _delta_t_constant , "deltaT_constant_5K" , "constant_mach"
566     ) ,
567     temperature_corrected_path_fuel(
568         profile_mach , _delta_t_linear , "deltaT_linear" , "constant_mach"
569     ) ,
570 ]
571 corrected_mach = pd.concat(corrected_mach_cases , ignore_index=True)
572 summary_mach = build_summary(standard_mach , corrected_mach , "constant_mach")
573
574 standard_all = pd.concat([standard_speed , standard_mach] , ignore_index=True)
575 corrected_all = pd.concat([corrected_speed , corrected_mach] , ignore_index=True)
576 summary_all = pd.concat([summary_speed , summary_mach] , ignore_index=True)

```



```

577
578 save_dataframe(standard_speed, results_dir / "problem2_path_fuel_standard.csv")
579 save_dataframe(corrected_speed, results_dir / "problem2_path_fuel_temperature_corrected.csv")
580 save_dataframe(summary_speed, results_dir / "problem2_temperature_summary.csv")
581 save_dataframe(standard_mach, results_dir / "problem2_path_fuel_standard_constant_mach.csv")
582 save_dataframe(
583     corrected_mach, results_dir / "problem2_path_fuel_temperature_corrected_constant_mach.csv")
584 )
585 save_dataframe(summary_mach, results_dir / "problem2_temperature_summary_constant_mach.csv")
586 save_dataframe(standard_all, results_dir / "problem2_path_fuel_standard_all_strategies.csv")
587 save_dataframe(
588     corrected_all, results_dir / "problem2_path_fuel_temperature_corrected_all_strategies.csv")
589 )
590 save_dataframe(summary_all, results_dir / "problem2_temperature_summary_all_strategies.csv")
591
592 make_figures(standard_speed, corrected_speed, figures_dir)
593 make_figures(standard_mach, corrected_mach, figures_dir, suffix="_constant_mach")
594 make_strategy_comparison_figures(standard_all, corrected_all, summary_all, figures_dir)
595 write_explanation(summary_all, base_dir / "explanation.md")
596 print("Problem 2 complete.")
597
598
599 if __name__ == "__main__":
600     main()

```